Chapter 1 — Mental Model: How Everything Fits Together

> **Goal of this chapter:** Before touching any code, understand *why* the architecture is structured the way it is. A developer who understands the "why" makes correct decisions independently. A developer who only knows the "what" copies patterns blindly and breaks things when requirements change.

1.1 The Problem This Architecture Solves

Imagine you are building a page that:

- Fetches a list of students from a server
- Lets the user filter by year
- Lets the user click a row to edit a student's activity
- Lets the user delete multiple rows at once
- Shows a loading skeleton while data loads
- Shows an error message if the fetch fails
- Shows a "no results" message if the list is empty

A beginner might put all of this in one component — one big file with `useState` for everything, `fetch()` calls inside `useEffect`, and all the JSX in one `return`. This works for tiny apps. It becomes unmaintainable the moment requirements change.

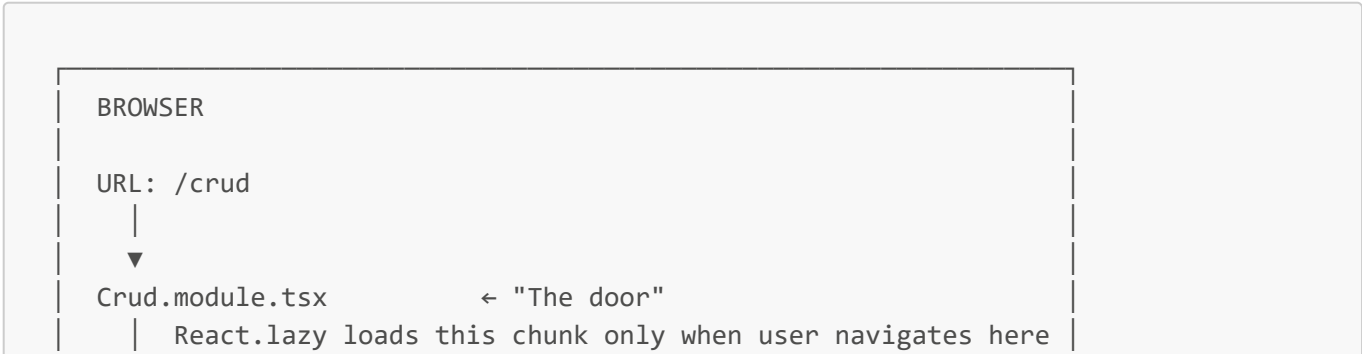
The problems with "everything in one file":

- You cannot test the data-fetching logic without rendering the UI
- You cannot reuse the table in another page
- Changing the API endpoint requires reading through 500 lines of mixed code
- Adding a column requires understanding the entire component
- Two developers cannot work on the same feature simultaneously without conflicts

The solution: Separation of Concerns (SoC)

Each file has exactly one responsibility. Changes are isolated. Testing is easy. Multiple developers can work in parallel.

1.2 The Layer Diagram



```
│    │                                                                  │
│    ▼                                                                  │
│  Crud.page.tsx            ← "The director"                            │
│    │  Calls useCrudPage(). Renders children. Zero logic.              │
│    │                                                                  │
│    ├──► useCrudPage.ts    ← "The brain"                               │
│    │      useReducer()    ← modal open/close state                    │
│    │      useState()      ← server filter params                      │
│    │      useGetItems()   ← list data from server                     │
│    │      useGetYears()   ← year dropdown options                     │
│    │      useMemo()       ← rows (DTO → Row mapping)                  │
│    │      useCallback()   ← stable handler references                 │
│    │                                                                  │
│    ├──► <ItemTable>       ← "The display"                             │
│    │      useReactTable() ← sort, filter, paginate                    │
│    │      Local state: sorting, search, columnFilters                 │
│    │                                                                  │
│    ├──► <ItemModal>       ← "The editor"                              │
│    │      useForm()       ← form state + Zod validation               │
│    │      FormProvider    ← shares form with ItemForm                 │
│    │      useCreateItem() ← POST mutation                             │
│    │      useUpdateItem() ← POST mutation                             │
│    │                                                                  │
│    │      └──► <ItemForm> ← "The fields"                              │
│    │             useFormContext() ← reads form from context           │
│    │             useGetCategories() ← self-fetches dropdown           │
│    │             Controller ← connects Shadcn inputs to RHF           │
│    │                                                                  │
│    └──► <DeleteConfirmModal> ← "The safety net"                       │
│           useDeleteItem()    ← DELETE mutation                        │
│           Promise.allSettled ← safe multi-delete                      │
│                                                                       │
└───────────────────────────────────────────────────────────────────────┘
```

1.3 The Data Flow — From Server to Screen

Understanding data flow is the most important mental model. Follow a single piece of data from the server to the screen:

```
C# Backend (NZQAController / NZAMPController)
    │
    │  HTTP GET /api/NZAMP/GetStudentsAttendanceSummary?year=10
    │  Returns: JSON array of StudentSummary objects (PascalCase)
    │
    ▼
crud.services.ts → getItems(params)
    │  apiClient.get(url, { params })
    │  zodParse(GetItemsResponseSchema, response.data)  ← validates at boundary
    │  Returns: ItemDto[]  (TypeScript type, still PascalCase)
    │
```

```
    ▼
crud.queries.ts → useGetItems(params)
    │  useQuery({ queryFn: () => getItems(params) })
    │  Caches result by query key
    │  Returns: { data: ItemDto[], isLoading, isFetching, isError }
    │
    ▼
useCrudPage.ts
    │  const { data: itemsResponse } = useGetItems(serverParams)
    │  const items = useMemo(() => itemsResponse ?? [], [itemsResponse])
    │  const rows = useMemo(() => items.map(mapItemDtoToRow), [items])
    │  Returns rows to the page
    │
    ▼
mapItemDtoToRow.ts (pure function)
    │  dto.StudentName → row.name        (PascalCase → camelCase)
    │  dto.LastActivityDate → row.lastActivityDate  (ISO string → "DD/MM/YYYY")
    │  dto.StudentNumber → row.id        (domain name → generic table name)
    │
    ▼
Crud.page.tsx
    │  <ItemTable rows={rows} ... />
    │
    ▼
ItemTable.tsx
    │  useReactTable({ data: rows, columns })
    │  Sorts, filters, paginates client-side
    │  Renders <Table> with flexRender()
    │
    ▼
Screen: User sees a sorted, paginated table of students
```

Key insight: Each step transforms the data into the format most natural for the next step. The server sends PascalCase. The mapper converts to camelCase. The table renders strings. Each layer is ignorant of the others' concerns.

1.4 The Mutation Flow — From User Action to Server Update

Now follow what happens when the user clicks "Edit" and saves:

```
User clicks "Edit" button on a row
    │
    ▼
ItemTable.tsx → actions.quick[0].onClick(row)
    │  Calls onEdit(row.id)  ← prop from Crud.page
    │
    ▼
Crud.page.tsx → handleEdit(itemId)
    │  (from useCrudPage)
    │
```

```
    ▼
useCrudPage.ts → handleEdit(itemId)
    │  items.find(i => i.StudentNumber === itemId)  ← finds the DTO
    │  buildEditContext(dto)  ← converts DTO to UI-ready context
    │  dispatch({ type: "OPEN_EDIT", ctx })  ← opens modal
    │
    ▼
buildEditContext.ts (pure function)
    │  dto.LastActivityDate → new Date(...)  (string → Date for date picker)
    │  dto.StudentNumber → studentId
    │  mode: "update"
    │
    ▼
Crud.page.tsx renders: <ItemModal open={true} editCtx={ctx} />
    │
    ▼
ItemModal.tsx
    │  useForm({ defaultValues: buildDefaults(editCtx) })
    │  reset(defaultValues) on open  ← pre-populates form
    │  User edits fields and clicks "Save Changes"
    │  handleSubmit(onSubmit) runs Zod validation
    │  If valid: updateMutation.mutate(request, { onSuccess: onClose })
    │
    ▼
crud.queries.ts → useUpdateItem → updateItem(payload)
    │
    ▼
crud.services.ts → updateItem(payload)
    │  apiClient.post("/api/NZAMP/UpdateStudentActivity", payload)
    │
    ▼
onSuccess callback:
    │  toast.success("Activity updated.")
    │  queryClient.invalidateQueries({ queryKey: QUERY_KEYS.items })
    │    ← marks cache stale → triggers re-fetch → table updates automatically
    │  onClose()  ← closes modal
    │
    ▼
Screen: Modal closes. Table re-fetches and shows updated data.
```

1.5 Why Each Layer Exists

`crud.dtos.ts` — The Contract

Problem it solves: The server can send anything. Without validation, a `null` where you expected a `string` causes a crash 3 clicks later with no useful error message.

Solution: Zod schemas validate the API response at the boundary. If the server sends wrong data, you get an immediate, clear error: `"Expected string, received null at path: StudentName at getItems"`.

Best practice followed: *Fail fast at the boundary, not silently in the middle of the UI.*

`crud.services.ts` — The HTTP Layer

Problem it solves: If you put `fetch()` calls directly in components, you cannot reuse them, test them, or change the URL without hunting through every component.

Solution: One function per endpoint. Pure async functions. No React, no UI.

Best practice followed: *Single Responsibility Principle — a function does one thing.*

`crud.queries.ts` — The Cache Layer

Problem it solves: Without caching, every component that needs the same data makes a separate network request. The user sees loading spinners everywhere. Data goes stale silently.

Solution: React Query caches by query key. All components sharing the same key share the same data. Mutations automatically invalidate the cache, triggering re-fetches.

Best practice followed: *Server state management is a solved problem — use a library (React Query) instead of reinventing it with `useState` + `useEffect`.*

`helpers/` — The Translation Layer

Problem it solves: The API shape (PascalCase, ISO dates, raw IDs) is different from what the UI needs (camelCase, formatted dates, display-friendly values). Mixing this translation into components makes them hard to test and reuse.

Solution: Pure functions that take one shape and return another. No side effects. Trivially testable.

Best practice followed: *Pure functions are the easiest code to test and reason about.*

`columns/itemColumns.tsx` — The Column Config

Problem it solves: If column definitions are inside the table component, adding a column requires understanding the entire component. Column definitions mixed with rendering logic are hard to read.

Solution: Declarative column config in a separate file. To add a column, edit only this file.

Best practice followed: *Declarative over imperative — describe what you want, not how to render it.*

`hooks/useCrudPage.ts` — The Brain

Problem it solves: If state and logic live in the page component, you cannot test them without rendering the UI. The page component becomes a 300-line mess.

Solution: Extract all state and logic into a custom hook. The page component becomes a thin orchestrator.

Best practice followed: *Extract logic into custom hooks — they are testable with `renderHook` without any UI.*

`Crud.page.tsx` — The Orchestrator

Problem it solves: If the page component contains business logic, it is hard to read, hard to test, and hard to change.

Solution: The page only calls the hook and renders children. It is the "wiring diagram" of the feature.

Best practice followed: *The page should be readable at a glance — you should understand the entire feature structure in under a minute.*

`Crud.module.tsx` — The Lazy Boundary

Problem it solves: Without code splitting, all feature code is bundled into one large file. The user downloads code for features they may never visit.

Solution: `React.lazy` defers loading until the user navigates to the route.

Best practice followed: *Code splitting — load only what the user needs, when they need it.*

1.6 The "One Job" Rule in Practice

Every time you write code, ask: **"What is this file's one job?"**

If you find yourself writing:

- An API call inside a component → move it to `services.ts`
- State management inside a component → move it to the hook
- Column definitions inside the table component → move them to `columns/`
- Business logic inside the page → move it to the hook
- Data transformation inside a query hook → move it to `helpers/`

This is not bureaucracy. It is the difference between code that is maintainable for years and code that becomes a nightmare in months.

1.7 React Fundamentals You Must Understand

Before reading the other chapters, make sure you understand these concepts:

Components

A React component is a function that returns JSX (HTML-like syntax). React calls this function every time state changes and updates the DOM.

```
// This is a component
function MyComponent() {
  return <div>Hello World</div>;
}
```

State

State is data that, when changed, causes React to re-render the component.

```
const [count, setCount] = useState(0);
// When setCount is called, React re-runs the component function
```

Props

Props are data passed from a parent component to a child component.

```
// Parent passes data
<ItemTable rows={rows} onEdit={handleEdit} />

// Child receives it
function ItemTable({ rows, onEdit }: ItemTableProps) { ... }
```

Hooks

Hooks are functions that start with `use`. They can only be called at the top level of a component or another hook — never inside loops, conditions, or nested functions.

```
// CORRECT
function MyComponent() {
  const [value, setValue] = useState("");  // top level ✓
  return <input value={value} onChange={e => setValue(e.target.value)} />;
}

// WRONG — hook inside a condition
function MyComponent() {
  if (someCondition) {
    const [value, setValue] = useState("");  // ✗ breaks React's rules
  }
}
```

The Re-render Cycle

Every time state changes, React re-runs the component function from top to bottom. This is why `useMemo` and `useCallback` exist — to prevent unnecessary work on every re-render.

Next: Chapter 2 — DTOs and Zod